

Computer Architecture Assignment

Branch Prediction and Cache Replacement in ChampSim

Akshar Ananya Varma
2023B5A80799H

Setup

The tests use ChampSim on WSL2 (Ubuntu 20.04, gcc 9.4). Each test has 1 million warmup instructions followed by 10 million measured instructions. The L1 instruction and data caches are 32 KB (8-way), the L2 cache is 256 KB (8-way), and the last-level cache (LLC) is 4 MB (16-way). All caches use 64-byte lines. Task 2 changes the LLC settings.

The single-core tests use four traces: 401.bzip2, 403.gcc, 456.hmmer, and 605.mcf. A Python script randomly selected the four-core mix using seed 42: 619.lbm, 403.gcc, 401.bzip2, and 620.omnetpp. Each core uses the listed cache size. Execution time is shown in simulated cycles because this value does not depend on the speed or load of the host computer.

1 Task 1: Branch Prediction

1.1 (a) Baseline Predictors

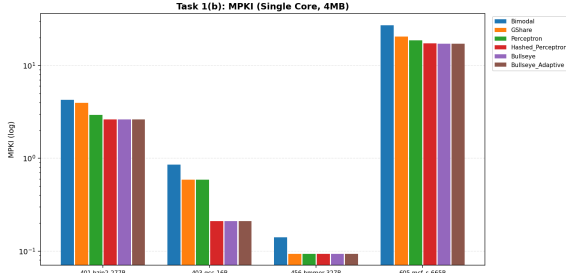
ChampSim includes four branch predictors: Bimodal, GShare, Perceptron, and Hashed Perceptron. Each predictor was compiled into a separate program using the build script. For example, the Hashed Perceptron command was `./build_champsim.sh hashed_perceptron no no no no lru 1`. Each program was then run on all four traces, and the results were used in part (b).

1.2 (b) Comparison

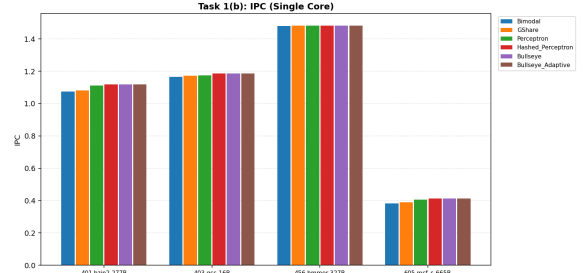
The predictors were tested on both single-core traces and the four-core mix. The comparison uses MPKI (wrong predictions per 1000 instructions), accuracy, prediction delay, and execution time. The values below come from the ChampSim output files.

Table 1: Single-core results (LLC 4 MB). Execution time is simulated cycles.

Predictor	Trace	MPKI	Accuracy (%)	IPC	Exec (cycles)
Bimodal	401.bzip2	4.2577	97.28	1.0747	9 304 927
Bimodal	403.gcc	0.8526	99.57	1.1662	8 575 039
Bimodal	456.hmmmer	0.1402	99.67	1.4806	6 754 237
Bimodal	605.mcf	27.2101	89.11	0.3819	26 188 101
GShare	401.bzip2	3.9527	97.47	1.0815	9 246 548
GShare	403.gcc	0.5905	99.70	1.1721	8 531 632
GShare	456.hmmmer	0.0935	99.78	1.4819	6 748 146
GShare	605.mcf	20.5604	91.77	0.3903	25 618 591
Perceptron	401.bzip2	2.9328	98.13	1.1120	8 992 635
Perceptron	403.gcc	0.5891	99.70	1.1741	8 516 934
Perceptron	456.hmmmer	0.0935	99.78	1.4819	6 748 146
Perceptron	605.mcf	18.7367	92.50	0.4047	24 706 986
Hashed Perceptron	401.bzip2	2.6127	98.33	1.1189	8 937 667
Hashed Perceptron	403.gcc	0.2104	99.89	1.1871	8 423 908
Hashed Perceptron	456.hmmmer	0.0935	99.78	1.4819	6 748 146
Hashed Perceptron	605.mcf	17.2468	93.10	0.4128	24 223 082
Bullseye	401.bzip2	2.6142	98.33	1.1186	8 939 537
Bullseye	403.gcc	0.2104	99.89	1.1871	8 423 908
Bullseye	456.hmmmer	0.0935	99.78	1.4819	6 748 146
Bullseye	605.mcf	17.2068	93.11	0.4130	24 214 615
Bullseye-Adapt	401.bzip2	2.6157	98.33	1.1186	8 939 746
Bullseye-Adapt	403.gcc	0.2104	99.89	1.1871	8 423 908
Bullseye-Adapt	456.hmmmer	0.0935	99.78	1.4819	6 748 146
Bullseye-Adapt	605.mcf	17.1933	93.12	0.4130	24 211 439



(a) MPKI



(b) IPC

Figure 1: Single-core predictor comparison.

The next table shows accuracy, MPKI, IPC, and execution time for each core in the four-core mix. The cores are 0:lbn, 1:gcc, 2:bzip2, and 3:omnetpp.

Table 2: Four-core results per core (seed-42 mix). Exec is simulated cycles.

Predictor	Core (trace)	MPKI	Acc (%)	IPC	Exec (cycles)
Bimodal	0 (lbm)	0.0313	99.70	0.8364	11 955 382
Bimodal	1 (gcc)	0.7258	99.63	1.1486	8 706 401
Bimodal	2 (bzip2)	14.7455	90.82	0.9977	10 022 796
Bimodal	3 (omnetpp)	8.7938	95.76	0.2567	38 950 997
GShare	0 (lbm)	0.0105	99.90	0.8372	11 944 775
GShare	1 (gcc)	0.4623	99.76	1.1550	8 657 794
GShare	2 (bzip2)	13.6884	91.47	1.0138	9 864 400
GShare	3 (omnetpp)	7.0099	96.62	0.2642	37 844 437
Perceptron	0 (lbm)	0.0106	99.90	0.8341	11 989 636
Perceptron	1 (gcc)	0.3913	99.80	1.1571	8 642 625
Perceptron	2 (bzip2)	11.4536	92.87	1.0444	9 574 508
Perceptron	3 (omnetpp)	5.0115	97.59	0.2708	36 929 691
Hashed Perceptron	0 (lbm)	0.0105	99.90	0.8308	12 036 151
Hashed Perceptron	1 (gcc)	0.1066	99.95	1.1696	8 550 092
Hashed Perceptron	2 (bzip2)	10.0868	93.73	1.0489	9 533 566
Hashed Perceptron	3 (omnetpp)	3.7869	98.18	0.2740	36 489 777
Bullseye	0 (lbm)	0.0106	99.90	0.8321	12 018 363
Bullseye	1 (gcc)	0.1067	99.95	1.1703	8 545 195
Bullseye	2 (bzip2)	10.0816	93.73	1.0489	9 534 164
Bullseye	3 (omnetpp)	3.7482	98.19	0.2745	36 429 190
Bullseye-Adapt	0 (lbm)	0.0105	99.90	0.8329	12 006 596
Bullseye-Adapt	1 (gcc)	0.1065	99.95	1.1697	8 548 990
Bullseye-Adapt	2 (bzip2)	10.0866	93.73	1.0516	9 509 456
Bullseye-Adapt	3 (omnetpp)	3.6390	98.25	0.2744	36 439 879

Results: performance improves from Bimodal to GShare, Perceptron, and Hashed Perceptron. The improvement is clear on difficult traces. For example, the MPKI for 605.mcf falls from 27.2 with Bimodal to 17.2 with Hashed Perceptron. The difference is small on gcc and hmmer because all predictors are already very accurate on these traces.

The four-core cumulative IPC (sum of the four per-core IPCs, i.e. total system throughput) is 3.239 for Bimodal, 3.270 for GShare, 3.306 for Perceptron, 3.323 for Hashed Perceptron, 3.326 for Bullseye, and 3.329 for Bullseye-Adapt. The adaptive version therefore gives the highest aggregate four-core throughput.

Fewer wrong predictions also reduce execution time. The stronger predictors finish 605.mcf in about 24.2 million cycles, compared with 26.2 million cycles for Bimodal. ChampSim does not measure prediction delay because it assumes that predictions take no time. The delay can only be estimated from the amount of work each predictor performs, summarised below.

Table 3: Estimated prediction cost (ChampSim does not report latency directly).

Predictor	Main hardware operation	Relative cost
Bimodal	one table lookup	lowest
GShare	XOR (history, PC) + one lookup	low
Perceptron	dot product over one weight vector	medium
Hashed Perceptron	16 table lookups + sum	high
Bullseye	Hashed Perceptron + occasional mini-perceptron	slightly higher
Bullseye-Adapt	Bullseye + threshold update	slightly higher

Bimodal is the cheapest (one lookup) and Hashed Perceptron the most expensive (16 lookups plus a sum). The Bullseye variants run alongside the Hashed Perceptron base and only add a small mini-perceptron on the few hard-to-predict branches, so they add little delay over the base predictor.

1.3 (c) Modified Bullseye and Adaptive Version

The modified Bullseye predictor follows the given flowchart and uses Hashed Perceptron as its main predictor. It counts the wrong predictions for each branch. A branch that is often predicted incorrectly is added to a small hard-to-predict (H2P) table and given a small perceptron of its own. This small perceptron can replace the main prediction, but only after it proves that it is more accurate for that branch. This prevents Bullseye from reducing accuracy on easy branches.

The adaptive version changes its limits while the program runs. The limits depend on the number of branches already in the H2P table:

$$\text{Exec} \geq 512 + 64N, \quad \text{Mispred} \geq 32 + 8N, \quad \text{Acc} < f(N),$$

where N is the number of branches in the H2P table. When the table is nearly empty, the limits are low so that hard branches can be found. As the table fills, the limits increase so that only the most difficult branches are added.

Results: the adaptive version has the lowest MPKI on the hardest trace: 17.19 on 605.mcf. It also has the best average result for the four-core mix. The improvement is small because Bullseye only helps when a trace contains a few very difficult branches. On easy traces, it behaves like the main predictor. The changing limits help it find difficult branches that the fixed version may miss.

2 Task 2: Last-Level Cache

2.1 (a) LLC Size (1–8 MB)

The LLC was tested at 1, 2, 4, and 8 MB to see how its size affects performance. Each version used 16-way associativity and LRU replacement and was run on all four traces.

Table 4: IPC vs LLC size, single core.

Trace	1 MB	2 MB	4 MB	8 MB
401.bzip2	0.6820	0.9199	1.1189	1.1189
403.gcc	1.1888	1.1888	1.1888	1.1888
456.hmmer	1.4032	1.4924	1.4924	1.4924
605.mcf	0.3418	0.3861	0.4128	0.4253

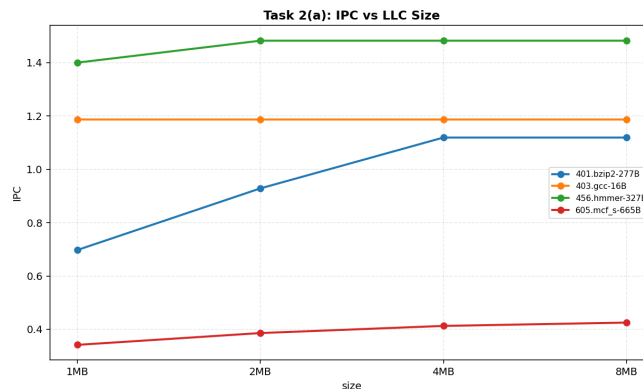


Figure 2: IPC vs LLC size.

Results: 605.mcf becomes faster as the LLC grows to 8 MB because it uses a large amount of memory. The performance of 401.bzip2 stops improving after 4 MB because its data fits in the cache at that size. gcc and hmmer do not improve because their data already fits in 1 MB.

2.2 (b) Associativity (16-way vs 8-way)

The 4 MB LLC was changed from 16-way to 8-way and run on all four traces. This test shows whether allowing fewer blocks in each cache set affects performance.

Table 5: IPC at 4 MB, 16-way vs 8-way.

Trace	16-way	8-way
401.bzip2	1.1189	1.1189
403.gcc	1.1888	1.1888
456.hmmer	1.4924	1.4924
605.mcf	0.4128	0.4113

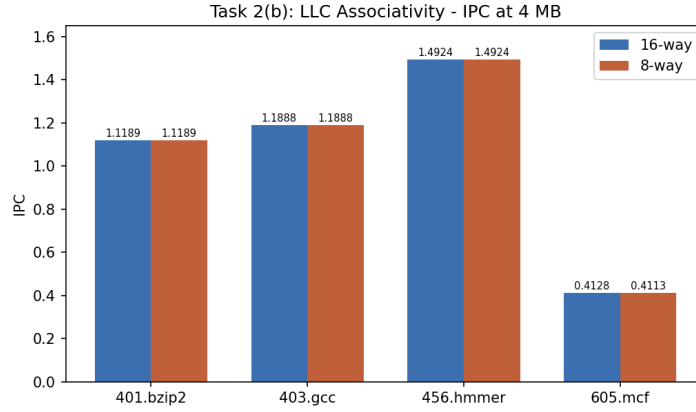


Figure 3: LLC associativity: IPC at 4 MB, 16-way vs 8-way.

Results: changing from 16-way to 8-way makes almost no difference at 4 MB. Only 605.mcf changes, and its IPC falls by less than 0.4%. This shows that most misses happen because the cache is too small, not because too many blocks map to the same set.

2.3 (c) MRU and Random Policies

Two new replacement policies, MRU and Random, were implemented as `.llc_repl` files. **MRU** removes the block used most recently, which is the opposite of LRU. It is included as a poor policy for comparison. **Random** removes a randomly chosen block and does not track previous accesses. The code for both policies is included in the appendix.

2.4 (d) Policy Comparison

All seven replacement policies (LRU, SRRIP, DRRIP, SHIP, MRU, Random, and the HCIT policy from part e) were tested at every LLC size with one and four cores. The comparison uses IPC, cache miss rates, LLC hit rate, LLC writes, and execution time. The tables show IPC at each cache size and all measurements at 4 MB. All tests use 64-byte blocks because ChampSim cannot easily change the block size for only the LLC.

Table 6: IPC by policy and size, 605.mcf single core.

Size	LRU	SRRIP	DRRIP	SHIP	MRU	Random	HCIT
1 MB	0.3418	0.3538	0.3482	0.3491	0.2609	0.3117	0.3393
2 MB	0.3861	0.3902	0.3851	0.3895	0.2783	0.3523	0.3905
4 MB	0.4128	0.4158	0.4115	0.4195	0.3229	0.3916	0.4155
8 MB	0.4253	0.4253	0.4242	0.4252	0.4153	0.4236	0.4253

The next table gives all measurements for 605.mcf. This is the only single-core trace affected by the replacement policy. The data used by 403.gcc, 456.hmmmer, and 401.bzip2 fits in the 4 MB cache, so all seven policies give the same results to four decimal places. These results are stored in the CSV files and are not repeated here. Execution time is shown in simulated cycles.

Table 7: All metrics by policy, 605.mcf at 4 MB (single core).

Policy	IPC	L1D miss%	L2 miss%	LLC miss%	LLC hit%	LLC writes	Exec (cyc)
LRU	0.4128	11.66	65.61	32.51	67.49	68040	24 223 082
SRRIP	0.4158	11.66	65.61	32.13	67.87	68040	24 049 326
DRRIP	0.4115	11.66	65.56	32.74	67.26	68070	24 301 201
SHIP	0.4195	11.66	65.60	31.74	68.26	68013	23 840 639
MRU	0.3229	11.67	65.61	58.45	41.55	68042	30 970 955
Random	0.3916	11.66	65.58	35.94	64.06	67975	25 536 240
HCIT	0.4155	11.66	65.61	32.17	67.83	68040	24 067 473

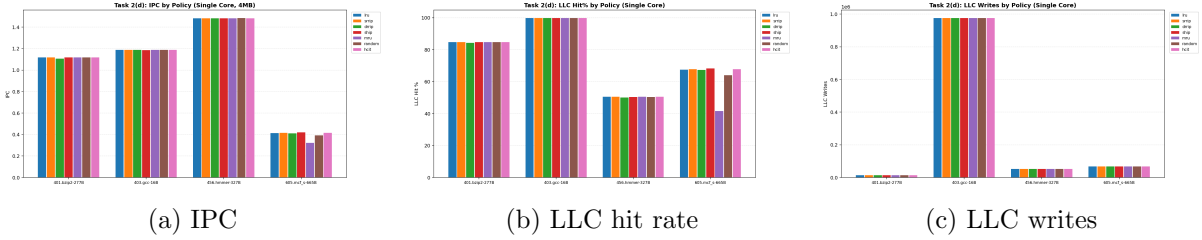


Figure 4: Policy comparison, single core (4 MB).

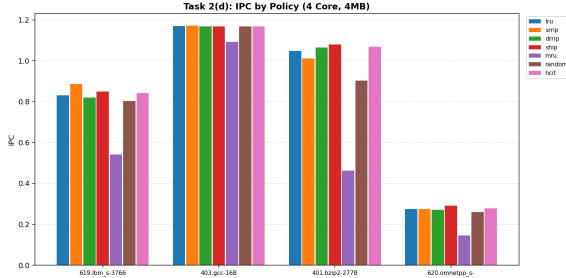
Table 8: Per-core IPC scaling with LLC size (4-core run, bzip2 core shown as representative).

Size	LRU	SRRIP	DRRIP	SHIP	MRU	Random	HCIT
1 MB	0.3714	0.3247	0.3909	0.3981	0.1114	0.2793	0.2478
2 MB	0.6061	0.5123	0.7776	0.8267	0.1493	0.4254	0.7370
4 MB	1.0489	1.0107	1.0658	1.0806	0.4625	0.9038	1.0693
8 MB	1.1209	1.1209	1.1202	1.1218	1.1193	1.1213	1.1213

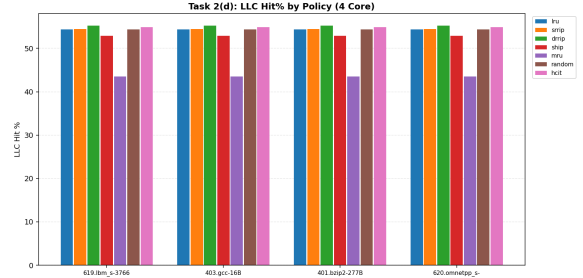
In the four-core test, all cores share the LLC. The LLC miss rate, hit rate, and write count therefore apply to the whole test rather than to each core. The L1D and L2 rates are measured per core, but they are almost the same for all cores. While Table 8 tracks one core across sizes, the table below summarises the whole four-core run at the 4 MB spec size: the shared-LLC metrics and the total cycles across all four cores.

Table 9: Four-core shared-LLC metrics at 4 MB.

Policy	LLC miss%	LLC hit%	LLC writes	Exec (cyc, sum)
LRU	42.64	57.36	90522	66 609 586
SRRIP	44.29	55.71	90418	66 030 872
DRRIP	42.85	57.15	90450	67 035 198
SHIP	42.08	57.92	90542	63 864 906
MRU	71.63	28.37	90349	118 089 014
Random	47.05	52.95	90469	70 570 100
HCIT	43.19	56.81	90467	65 775 249



(a) IPC (4 core)



(b) LLC hit rate (4 core)

Figure 5: Policy comparison, 4 core (4 MB/core).

Results: SHIP gives the best overall performance, while MRU gives the worst. MRU often removes blocks that will soon be used again. Random performs between these two policies. The LLC policy has almost no effect on the L1 and L2 miss rates or the number of LLC writes. The upper caches are checked before the LLC, and writes are mainly set by the program’s store instructions.

A higher LLC hit rate usually gives a shorter execution time. SHIP completes the four-core test fastest, while MRU is the slowest. All policies give the same result on gcc and hammer because their data fits in the cache. The results also become very similar at 8 MB because most of the data fits at that size.

2.5 (e) HCIT — Bullseye-Inspired Cache Policy

HCIT uses the Bullseye idea that a small number of instructions may cause many cache misses. It adds three parts to SRRIP: a table, a protection rule, and an admission rule.

- **Table:** A 64-set, 8-way table uses the instruction address (PC) as its index. It counts accesses, misses, inserted blocks, and reused blocks for each PC.
- **Protection:** Blocks from a protected PC are inserted with RRPV 0 so that they remain in the cache longer. The policy also tries not to remove these blocks. Other blocks are handled normally.
- **Admission:** A PC is protected only when it causes enough misses and its blocks are reused. The limits rise as more PCs are protected, which keeps the protected group small. A PC loses protection if its blocks are no longer reused.

HCIT checks whether a block was reused when that block leaves the cache. Using only the miss rate could protect instructions that read a stream of data only once. Protecting such blocks would waste cache space. Instead, a flag records whether each block received a hit before it was removed. SHIP uses a similar method.

Results: on 605.mcf, HCIT reaches an IPC of 0.4155, compared with 0.4128 for LRU. It is also close to SHIP, the best policy in this test. HCIT protects no PCs on the easy traces

and gives the same results as SRRIP. This is expected because those traces do not contain a small group of difficult PCs. On 605.mcf, HCIT protects only one PC, but this is enough to beat LRU. This supports the idea that a very small group can cause many cache misses. The advantage disappears at 8 MB because the data fits in the larger cache.

3 Conclusion

Across both tasks the same principle recurs: extra machinery helps only where the underlying problem exists. For branch prediction, Hashed Perceptron is the strongest baseline, and the adaptive Bullseye lowers MPKI on the hardest trace (605.mcf, 17.19) and gives the best aggregate four-core throughput, while behaving like the base predictor on easy traces. For the last-level cache, a larger LLC helps only memory-intensive workloads such as 605.mcf, and associativity barely matters once the cache is capacity-limited. Among replacement policies SHIP is the strongest overall; the proposed HCIT policy approaches SHIP by tracking only a small set of hard-to-cache PCs, which confirms the Bullseye premise that a small subset is responsible for most of the misses.

A bullseye.bpred

```

1 // Modified Bullseye Branch Predictor
2 // Base: hashed perceptron (per assignment flowchart)
3 // H2P subsystem inspired by Behrendt, Pun, Nair, "Taming Wild Branches", CBP 2025
4
5 #include <stdio.h>
6 #include <string.h>
7 #include <math.h>
8 #include <stdlib.h>
9 #include "ooo_cpu.h"
10
11 /* ----- BASE: hashed perceptron ----- */
12 #define NTABLES 16
13 #define MAXHIST 232
14 #define MINHIST 3
15 #define SPEED 18
16 int history_lengths[NTABLES] = {0,3,4,6,8,10,14,19,26,36,49,67,91,125,170,MAXHIST};
17 #define LOG_TABLE_SIZE 12
18 #define TABLE_SIZE (1<<LOG_TABLE_SIZE)
19 #define NGHIST_WORDS (MAXHIST/LOG_TABLE_SIZE+1)
20
21 int tables[NUM_CPUS][NTABLES][TABLE_SIZE];
22 unsigned int ghist_words[NUM_CPUS][NGHIST_WORDS];
23 unsigned int indices[NUM_CPUS][NTABLES];
24 int theta[NUM_CPUS], tc[NUM_CPUS], yout[NUM_CPUS];
25
26 /* ----- HIT: Hard-to-Predict Identification Table ----- */
27 #define HIT_SETS 64
28 #define HIT_WAYS 8
29 #define EXEC_THRESHOLD 2048
30 #define MISPRED_THRESHOLD 256
31 #define ACC_THRESHOLD 0.95
32
33 struct hit_entry_t {
34     uint64_t tag;
35     uint32_t exec;
36     uint32_t mispred;
37     uint8_t valid;
38     uint8_t h2p_active;
39     uint32_t lru;
40 };
41 hit_entry_t hit_table[NUM_CPUS][HIT_SETS][HIT_WAYS];
42 uint32_t hit_lru_clock[NUM_CPUS];
43
44 /* ----- Mini perceptron layer ----- */
45 #define MAX_H2P 32

```



```

46 #define LOCAL_HIST_LEN 24
47 #define MINI_WEIGHTS (LOCAL_HIST_LEN+1)
48 #define MINI_THETA (int)(1.93*LOCAL_HIST_LEN+14)
49 #define WARMUP_OCC 512
50 #define WINRATE_MAX 64
51 #define WINRATE_GATE 8
52
53 struct h2p_entry_t {
54     uint64_t pc;
55     uint8_t valid;
56     int weights[MINI_WEIGHTS];
57     uint32_t local_hist;
58     uint32_t occurrences;
59     uint32_t mini_correct;
60     uint32_t mini_total;
61     int32_t winrate; // saturating: mini wins vs base wins
62 };
63 h2p_entry_t h2p[NUM_CPUS][MAX_H2P];
64 int h2p_fifo_next[NUM_CPUS];
65 int n_h2p[NUM_CPUS];
66
67 /* per-prediction saved state */
68 int s_h2p_idx[NUM_CPUS];
69 int s_mini_out[NUM_CPUS];
70 uint8_t s_base_pred[NUM_CPUS];
71 uint8_t s_used_mini[NUM_CPUS];
72
73 /* stats */
74 uint64_t stat_overrides[NUM_CPUS];
75 uint64_t stat_h2p_admitted[NUM_CPUS];
76 uint64_t stat_override_right[NUM_CPUS];
77 uint64_t stat_override_wrong[NUM_CPUS];
78 uint64_t stat_base_would_be_right[NUM_CPUS];
79 uint64_t stat_total_branches[NUM_CPUS];
80
81 static inline uint32_t hit_set_index(uint64_t pc) { return (pc >> 2) & (HIT_SETS-1); }
82 static inline uint64_t hit_tag(uint64_t pc) { return (pc >> 2) >> 6; }
83
84 static int find_h2p(uint32_t c, uint64_t pc) {
85     for (int i=0; i<MAX_H2P; i++)
86         if (h2p[c][i].valid && h2p[c][i].pc == pc) return i;
87     return -1;
88 }
89
90 static void admit_h2p(uint32_t c, uint64_t pc) {
91     int idx = h2p_fifo_next[c];
92     if (h2p[c][idx].valid) n_h2p[c]--;
93     memset(&h2p[c][idx], 0, sizeof(h2p_entry_t));
94     h2p[c][idx].pc = pc;
95     h2p[c][idx].valid = 1;
96     n_h2p[c]++;
97     stat_h2p_admitted[c]++;
98     h2p_fifo_next[c] = (idx+1) % MAX_H2P;
99 }
100
101 static int mini_predict(uint32_t c, int idx) {
102     h2p_entry_t *e = &h2p[c][idx];
103     int out = e->weights[0];
104     for (int i=0; i<LOCAL_HIST_LEN; i++) {
105         int x = ((e->local_hist >> i) & 1) ? 1 : -1;
106         out += e->weights[i+1] * x;
107     }
108     return out;
109 }
110
111 static void mini_update(uint32_t c, int idx, uint8_t taken, int out) {
112     h2p_entry_t *e = &h2p[c][idx];
113     int pred = (out >= 0) ? 1 : 0;
114     int a = (out < 0) ? -out : out;
115     if (pred != taken || a <= MINI_THETA) {
116         int t = taken ? 1 : -1;
117         if (e->weights[0]*t < 127) e->weights[0] += t;
118         for (int i=0; i<LOCAL_HIST_LEN; i++) {

```

```

119         int x = ((e->local_hist >> i) & 1) ? 1 : -1;
120         int upd = t * x;
121         int *w = &e->weights[i+1];
122         if (upd > 0) { if (*w < 127) (*w)++; }
123         else        { if (*w > -128) (*w)--; }
124     }
125 }
126 e->local_hist = ((e->local_hist << 1) | (taken?1:0)) & ((1u<<LOCAL_HIST_LEN)-1);
127 e->occurrences++;
128 e->mini_total++;
129 if (pred == taken) e->mini_correct++;
130 }
131
132 void O3_CPU::initialize_branch_predictor() {
133     cout << "CPU " << cpu << " Modified Bullseye branch predictor "
134         << "(base: hashed perceptron, H2P slots: " << MAX_H2P
135         << ", fixed thresholds)" << endl;
136     memset(tables, 0, sizeof(tables));
137     memset(ghist_words, 0, sizeof(ghist_words));
138     memset(hit_table, 0, sizeof(hit_table));
139     memset(h2p, 0, sizeof(h2p));
140     for (int i=0; i<NUM_CPUS; i++) {
141         theta[i] = 10; tc[i] = 0;
142         h2p_fifo_next[i] = 0; n_h2p[i] = 0;
143         hit_lru_clock[i] = 0;
144         stat_overrides[i] = 0; stat_h2p_admitted[i] = 0;
145         stat_override_right[i] = 0; stat_override_wrong[i] = 0;
146         stat_base_would_be_right[i] = 0; stat_total_branches[i] = 0;
147     }
148 }
149
150 uint8_t O3_CPU::predict_branch(uint64_t pc) {
151     /* --- base hashed perceptron --- */
152     yout[cpu] = 0;
153     for (int i=0; i<NTABLES; i++) {
154         int n = history_lengths[i];
155         unsigned int x = 0;
156         int most_words = n / LOG_TABLE_SIZE;
157         int last_word = n % LOG_TABLE_SIZE;
158         int j;
159         for (j=0; j<most_words; j++) x ^= ghist_words[cpu][j];
160         x ^= ghist_words[cpu][j] & ((1<<last_word)-1);
161         x ^= pc;
162         x &= TABLE_SIZE-1;
163         indices[cpu][i] = x;
164         yout[cpu] += tables[cpu][i][x];
165     }
166     uint8_t base_pred = (yout[cpu] >= 1);
167     s_base_pred[cpu] = base_pred;
168     s_used_mini[cpu] = 0;
169     s_h2p_idx[cpu] = -1;
170
171     /* --- H2P override path --- */
172     int idx = find_h2p(cpu, pc);
173     if (idx >= 0) {
174         s_h2p_idx[cpu] = idx;
175         int out = mini_predict(cpu, idx);
176         s_mini_out[cpu] = out;
177         h2p_entry_t *e = &h2p[cpu][idx];
178         int a = (out < 0) ? -out : out;
179         if (e->occurrences >= WARMUP_OCC && a > MINI_THETA/2 && e->winrate >=
180             WINRATE_GATE) {
181             s_used_mini[cpu] = 1;
182             stat_overrides[cpu]++;
183             return (out >= 0) ? 1 : 0;
184         }
185     }
186     return base_pred;
187 }
188
189 void O3_CPU::last_branch_result(uint64_t pc, uint8_t taken) {
190     bool base_correct = (taken == s_base_pred[cpu]);

```

```

191 stat_total_branches[cpu]++;
192 if (s_used_mini[cpu]) {
193     uint8_t mini_pred = (s_mini_out[cpu] >= 0) ? 1 : 0;
194     if (mini_pred == taken) stat_override_right[cpu]++;
195     else stat_override_wrong[cpu]++;
196     if (base_correct) stat_base_would_be_right[cpu]++;
197 }
198 if ((stat_total_branches[cpu] % 1000000) == 0) {
199     cout << "[BULLSEYE cpu" << cpu << "] branches=" << stat_total_branches[cpu]
200         << " admitted=" << stat_h2p_admitted[cpu]
201         << " active=" << n_h2p[cpu]
202         << " overrides=" << stat_overrides[cpu]
203         << " ovr_right=" << stat_override_right[cpu]
204         << " ovr_wrong=" << stat_override_wrong[cpu]
205         << " base_right_when_ovr=" << stat_base_would_be_right[cpu]
206         << endl;
207 }
208
209 /* --- HIT bookkeeping --- */
210 uint32_t set = hit_set_index(pc);
211 uint64_t tag = hit_tag(pc);
212 int way = -1;
213 for (int w=0; w<HIT_WAYS; w++)
214     if (hit_table[cpu][set][w].valid && hit_table[cpu][set][w].tag == tag) { way = w
        ; break; }
215 if (way < 0) {
216     int victim = 0; uint32_t oldest = 0xFFFFFFFF;
217     for (int w=0; w<HIT_WAYS; w++) {
218         if (!hit_table[cpu][set][w].valid) { victim = w; break; }
219         if (hit_table[cpu][set][w].lru < oldest) { oldest = hit_table[cpu][set][w].
            lru; victim = w; }
220     }
221     memset(&hit_table[cpu][set][victim], 0, sizeof(hit_entry_t));
222     hit_table[cpu][set][victim].tag = tag;
223     hit_table[cpu][set][victim].valid = 1;
224     way = victim;
225 }
226 hit_entry_t *h = &hit_table[cpu][set][way];
227 h->lru = ++hit_lru_clock[cpu];
228 h->exec++;
229 if (!base_correct) h->mispred++;
230
231 /* --- admission test (fixed thresholds) --- */
232 if (!h->h2p_active && h->exec >= EXEC_THRESHOLD && h->mispred >= MISPREP_THRESHOLD)
233     {
234         double acc = 1.0 - (double)h->mispred / (double)h->exec;
235         if (acc < ACC_THRESHOLD) {
236             h->h2p_active = 1;
237             admit_h2p(cpu, pc);
238         }
239     }
240
241 /* --- mini perceptron update + head-to-head scoring --- */
242 if (s_h2p_idx[cpu] >= 0 && h2p[cpu][s_h2p_idx[cpu]].valid &&
243     h2p[cpu][s_h2p_idx[cpu]].pc == pc) {
244     h2p_entry_t *e = &h2p[cpu][s_h2p_idx[cpu]];
245     uint8_t mini_pred = (s_mini_out[cpu] >= 0) ? 1 : 0;
246     bool mini_ok = (mini_pred == taken);
247     if (mini_ok && !base_correct) {
248         if (e->winrate < WINRATE_MAX) e->winrate++;
249     } else if (!mini_ok && base_correct) {
250         if (e->winrate > -WINRATE_MAX) e->winrate--;
251     }
252     mini_update(cpu, s_h2p_idx[cpu], taken, s_mini_out[cpu]);
253 }
254
255 /* --- base predictor update (always; no TAGE-filtering in modified version) --- */
256 bool b = taken;
257 for (int i=0; i<NGHIST_WORDS; i++) {
258     ghist_words[cpu][i] <= 1;
259     ghist_words[cpu][i] |= b;
260     b = !(ghist_words[cpu][i] & TABLE_SIZE);
261     ghist_words[cpu][i] &= TABLE_SIZE-1;

```

```

261     }
262     int a = (yout[cpu] < 0) ? -yout[cpu] : yout[cpu];
263     if (!base_correct || a < theta[cpu]) {
264         for (int i=0; i<NTABLES; i++) {
265             int *c = &tables[cpu][i][indices[cpu][i]];
266             if (taken) { if (*c < 127) (*c)++; }
267             else      { if (*c > -128) (*c)--; }
268         }
269         if (!base_correct) {
270             tc[cpu]++;
271             if (tc[cpu] >= SPEED) { theta[cpu]++; tc[cpu] = 0; }
272         } else if (a < theta[cpu]) {
273             tc[cpu]--;
274             if (tc[cpu] <= -SPEED) { theta[cpu]--; tc[cpu] = 0; }
275         }
276     }
277 }

```

B bullseye_adaptive.bpred

```

1  // Modified Bullseye Branch Predictor
2  // Base: hashed perceptron (per assignment flowchart)
3  // H2P subsystem inspired by Behrendt, Pun, Nair, "Taming Wild Branches", CBP 2025
4
5  #include <stdio.h>
6  #include <string.h>
7  #include <math.h>
8  #include <stdlib.h>
9  #include "ooo_cpu.h"
10
11  /* ----- BASE: hashed perceptron ----- */
12  #define NTABLES 16
13  #define MAXHIST 232
14  #define MINHIST 3
15  #define SPEED 18
16  int history_lengths[NTABLES] = {0,3,4,6,8,10,14,19,26,36,49,67,91,125,170,MAXHIST};
17  #define LOG_TABLE_SIZE 12
18  #define TABLE_SIZE (1<<LOG_TABLE_SIZE)
19  #define NGHIST_WORDS (MAXHIST/LOG_TABLE_SIZE+1)
20
21  int tables[NUM_CPUS][NTABLES][TABLE_SIZE];
22  unsigned int ghist_words[NUM_CPUS][NGHIST_WORDS];
23  unsigned int indices[NUM_CPUS][NTABLES];
24  int theta[NUM_CPUS], tc[NUM_CPUS], yout[NUM_CPUS];
25
26  /* ----- HIT: Hard-to-Predict Identification Table ----- */
27  #define HIT_SETS 64
28  #define HIT_WAYS 8
29  /* Adaptive thresholds (paper Eq. 1a-1d), scaled for ChampSim's
30     hashed-perceptron base rather than TAGE-SC-L. */
31  #define EXEC_BASE 512 // relaxed vs fixed 2048
32  #define EXEC_PER_H2P 64 // tighten as population grows
33  #define MISRPRED_BASE 32 // relaxed vs fixed 256
34  #define MISRPRED_PER_H2P 8
35
36  // f(N): accuracy ceiling tightens as N_H2P grows
37  static inline double acc_ceiling(int N) {
38      if (N < 8) return 0.995 - 0.005 * N / 8.0;
39      if (N <= 24) return 0.99 - 0.01 * (N - 8) / 16.0;
40      return 0.90;
41  }
42
43  struct hit_entry_t {
44      uint64_t tag;
45      uint32_t exec;
46      uint32_t mispred;
47      uint8_t valid;
48      uint8_t h2p_active;
49      uint32_t lru;
50  };

```

```

51 hit_entry_t hit_table[NUM_CPUS][HIT_SETS][HIT_WAYS];
52 uint32_t hit_lru_clock[NUM_CPUS];
53
54 /* ----- Mini perceptron layer ----- */
55 #define MAX_H2P 32
56 #define LOCAL_HIST_LEN 24
57 #define MINI_WEIGHTS (LOCAL_HIST_LEN+1)
58 #define MINI_THETA (int)(1.93*LOCAL_HIST_LEN+14)
59 #define WARMUP_OCC 512
60 #define WINRATE_MAX 64
61 #define WINRATE_GATE 8
62
63 struct h2p_entry_t {
64     uint64_t pc;
65     uint8_t valid;
66     int weights[MINI_WEIGHTS];
67     uint32_t local_hist;
68     uint32_t occurrences;
69     uint32_t mini_correct;
70     uint32_t mini_total;
71     int32_t winrate; // saturating: mini wins vs base wins
72 };
73 h2p_entry_t h2p[NUM_CPUS][MAX_H2P];
74 int h2p_fifo_next[NUM_CPUS];
75 int n_h2p[NUM_CPUS];
76
77 /* per-prediction saved state */
78 int s_h2p_idx[NUM_CPUS];
79 int s_mini_out[NUM_CPUS];
80 uint8_t s_base_pred[NUM_CPUS];
81 uint8_t s_used_mini[NUM_CPUS];
82
83 /* stats */
84 uint64_t stat_overrides[NUM_CPUS];
85 uint64_t stat_h2p_admitted[NUM_CPUS];
86 uint64_t stat_override_right[NUM_CPUS];
87 uint64_t stat_override_wrong[NUM_CPUS];
88 uint64_t stat_base_would_be_right[NUM_CPUS];
89 uint64_t stat_total_branches[NUM_CPUS];
90
91 static inline uint32_t hit_set_index(uint64_t pc) { return (pc >> 2) & (HIT_SETS-1); }
92 static inline uint64_t hit_tag(uint64_t pc) { return (pc >> 2) >> 6; }
93
94 static int find_h2p(uint32_t c, uint64_t pc) {
95     for (int i=0; i<MAX_H2P; i++)
96         if (h2p[c][i].valid && h2p[c][i].pc == pc) return i;
97     return -1;
98 }
99
100 static void admit_h2p(uint32_t c, uint64_t pc) {
101     int idx = h2p_fifo_next[c];
102     if (h2p[c][idx].valid) n_h2p[c]--;
103     memset(&h2p[c][idx], 0, sizeof(h2p_entry_t));
104     h2p[c][idx].pc = pc;
105     h2p[c][idx].valid = 1;
106     n_h2p[c]++;
107     stat_h2p_admitted[c]++;
108     h2p_fifo_next[c] = (idx+1) % MAX_H2P;
109 }
110
111 static int mini_predict(uint32_t c, int idx) {
112     h2p_entry_t *e = &h2p[c][idx];
113     int out = e->weights[0];
114     for (int i=0; i<LOCAL_HIST_LEN; i++) {
115         int x = ((e->local_hist >> i) & 1) ? 1 : -1;
116         out += e->weights[i+1] * x;
117     }
118     return out;
119 }
120
121 static void mini_update(uint32_t c, int idx, uint8_t taken, int out) {
122     h2p_entry_t *e = &h2p[c][idx];
123     int pred = (out >= 0) ? 1 : 0;

```

```

124     int a = (out < 0) ? -out : out;
125     if (pred != taken || a <= MINI_THETA) {
126         int t = taken ? 1 : -1;
127         if (e->weights[0]*t < 127) e->weights[0] += t;
128         for (int i=0; i<LOCAL_HIST_LEN; i++) {
129             int x = ((e->local_hist >> i) & 1) ? 1 : -1;
130             int upd = t * x;
131             int *w = &e->weights[i+1];
132             if (upd > 0) { if (*w < 127) (*w)++; }
133             else { if (*w > -128) (*w)--; }
134         }
135     }
136     e->local_hist = ((e->local_hist << 1) | (taken?1:0)) & ((1u<<LOCAL_HIST_LEN)-1);
137     e->occurrences++;
138     e->mini_total++;
139     if (pred == taken) e->mini_correct++;
140 }
141
142 void O3_CPU::initialize_branch_predictor() {
143     cout << "CPU " << cpu << " Modified Bullseye branch predictor "
144         << "(base: hashed perceptron, H2P slots: " << MAX_H2P
145         << ", ADAPTIVE thresholds)" << endl;
146     memset(tables, 0, sizeof(tables));
147     memset(ghist_words, 0, sizeof(ghist_words));
148     memset(hit_table, 0, sizeof(hit_table));
149     memset(h2p, 0, sizeof(h2p));
150     for (int i=0; i<NUM_CPUS; i++) {
151         theta[i] = 10; tc[i] = 0;
152         h2p_fifo_next[i] = 0; n_h2p[i] = 0;
153         hit_lru_clock[i] = 0;
154         stat_overrides[i] = 0; stat_h2p_admitted[i] = 0;
155         stat_override_right[i] = 0; stat_override_wrong[i] = 0;
156         stat_base_would_be_right[i] = 0; stat_total_branches[i] = 0;
157     }
158 }
159
160 uint8_t O3_CPU::predict_branch(uint64_t pc) {
161     /* --- base hashed perceptron --- */
162     yout[cpu] = 0;
163     for (int i=0; i<NTABLES; i++) {
164         int n = history_lengths[i];
165         unsigned int x = 0;
166         int most_words = n / LOG_TABLE_SIZE;
167         int last_word = n % LOG_TABLE_SIZE;
168         int j;
169         for (j=0; j<most_words; j++) x ^= ghist_words[cpu][j];
170         x ^= ghist_words[cpu][j] & ((1<<last_word)-1);
171         x ^= pc;
172         x &= TABLE_SIZE-1;
173         indices[cpu][i] = x;
174         yout[cpu] += tables[cpu][i][x];
175     }
176     uint8_t base_pred = (yout[cpu] >= 1);
177     s_base_pred[cpu] = base_pred;
178     s_used_mini[cpu] = 0;
179     s_h2p_idx[cpu] = -1;
180
181     /* --- H2P override path --- */
182     int idx = find_h2p(cpu, pc);
183     if (idx >= 0) {
184         s_h2p_idx[cpu] = idx;
185         int out = mini_predict(cpu, idx);
186         s_mini_out[cpu] = out;
187         h2p_entry_t *e = &h2p[cpu][idx];
188         int a = (out < 0) ? -out : out;
189         int gate = WINRATE_GATE + (n_h2p[cpu] / 4);
190         if (e->occurrences >= WARMUP_OCC/4 && a > MINI_THETA/2 && e->winrate >= gate) {
191             s_used_mini[cpu] = 1;
192             stat_overrides[cpu]++;
193             return (out >= 0) ? 1 : 0;
194         }
195     }
196     return base_pred;

```

```

197 }
198
199 void O3_CPU::last_branch_result(uint64_t pc, uint8_t taken) {
200     bool base_correct = (taken == s_base_pred[cpu]);
201
202     stat_total_branches[cpu]++;
203     if (s_used_mini[cpu]) {
204         uint8_t mini_pred = (s_mini_out[cpu] >= 0) ? 1 : 0;
205         if (mini_pred == taken) stat_override_right[cpu]++;
206         else stat_override_wrong[cpu]++;
207         if (base_correct) stat_base_would_be_right[cpu]++;
208     }
209     if ((stat_total_branches[cpu] % 1000000) == 0) {
210         cout << "[BULLSEYE-ADAPT cpu" << cpu << "]" branches=" << stat_total_branches[cpu]
211             << " admitted=" << stat_h2p_admitted[cpu]
212             << " active=" << n_h2p[cpu]
213             << " overrides=" << stat_overrides[cpu]
214             << " ovr_right=" << stat_override_right[cpu]
215             << " ovr_wrong=" << stat_override_wrong[cpu]
216             << " base_right_when_ovr=" << stat_base_would_be_right[cpu]
217             << endl;
218     }
219
220     /* --- HIT bookkeeping --- */
221     uint32_t set = hit_set_index(pc);
222     uint64_t tag = hit_tag(pc);
223     int way = -1;
224     for (int w=0; w<HIT_WAYS; w++)
225         if (hit_table[cpu][set][w].valid && hit_table[cpu][set][w].tag == tag) { way = w
226             ; break; }
227     if (way < 0) {
228         int victim = 0; uint32_t oldest = 0xFFFFFFFF;
229         for (int w=0; w<HIT_WAYS; w++) {
230             if (!hit_table[cpu][set][w].valid) { victim = w; break; }
231             if (hit_table[cpu][set][w].lru < oldest) { oldest = hit_table[cpu][set][w].
232                 lru; victim = w; }
233         }
234         memset(&hit_table[cpu][set][victim], 0, sizeof(hit_entry_t));
235         hit_table[cpu][set][victim].tag = tag;
236         hit_table[cpu][set][victim].valid = 1;
237         way = victim;
238     }
239     hit_entry_t *h = &hit_table[cpu][set][way];
240     h->lru = ++hit_lru_clock[cpu];
241     h->exec++;
242     if (!base_correct) h->mispred++;
243
244     /* --- admission test (fixed thresholds) --- */
245     uint32_t exec_th = EXEC_BASE + EXEC_PER_H2P * n_h2p[cpu];
246     uint32_t mispred_th = MISPREP_BASE + MISPREP_PER_H2P * n_h2p[cpu];
247     if (!h->h2p_active && n_h2p[cpu] < MAX_H2P &&
248         h->exec >= exec_th && h->mispred >= mispred_th) {
249         double acc = 1.0 - (double)h->mispred / (double)h->exec;
250         if (acc < acc_ceiling(n_h2p[cpu])) {
251             h->h2p_active = 1;
252             admit_h2p(cpu, pc);
253         }
254     }
255
256     /* --- mini perceptron update + head-to-head scoring --- */
257     if (s_h2p_idx[cpu] >= 0 && h2p[cpu][s_h2p_idx[cpu]].valid &&
258         h2p[cpu][s_h2p_idx[cpu]].pc == pc) {
259         h2p_entry_t *e = &h2p[cpu][s_h2p_idx[cpu]];
260         uint8_t mini_pred = (s_mini_out[cpu] >= 0) ? 1 : 0;
261         bool mini_ok = (mini_pred == taken);
262         if (mini_ok && !base_correct) {
263             if (e->winrate < WINRATE_MAX) e->winrate++;
264         } else if (!mini_ok && base_correct) {
265             if (e->winrate > -WINRATE_MAX) e->winrate--;
266         }
267         mini_update(cpu, s_h2p_idx[cpu], taken, s_mini_out[cpu]);
268     }

```

```

267 // adaptive eviction: base wins decisively -> release slot
268 if (e->occurrences > WARMUP_OCC && e->winrate <= -WINRATE_MAX/2) {
269     e->valid = 0;
270     n_h2p[cpu]--;
271 }
272 }
273
274 /* --- base predictor update (always; no TAGE-filtering in modified version) --- */
275 bool b = taken;
276 for (int i=0; i<NGHIST_WORDS; i++) {
277     ghist_words[cpu][i] <= 1;
278     ghist_words[cpu][i] |= b;
279     b = !(ghist_words[cpu][i] & TABLE_SIZE);
280     ghist_words[cpu][i] &= TABLE_SIZE-1;
281 }
282 int a = (yout[cpu] < 0) ? -yout[cpu] : yout[cpu];
283 if (!base_correct || a < theta[cpu]) {
284     for (int i=0; i<NTABLES; i++) {
285         int *c = &tables[cpu][i][indices[cpu][i]];
286         if (taken) { if (*c < 127) (*c)++; }
287         else { if (*c > -128) (*c)--; }
288     }
289     if (!base_correct) {
290         tc[cpu]++;
291         if (tc[cpu] >= SPEED) { theta[cpu]++; tc[cpu] = 0; }
292     } else if (a < theta[cpu]) {
293         tc[cpu]--;
294         if (tc[cpu] <= -SPEED) { theta[cpu]--; tc[cpu] = 0; }
295     }
296 }
297 }

```

C mru.llc_repl

```

1 #include "cache.h"
2 // MRU replacement: evict the most recently used block.
3 // block[set][way].lru == 0 is the MRU position.
4
5 void CACHE::llc_initialize_replacement()
6 {
7     cout << "Initialize LLC MRU replacement policy" << endl;
8 }
9
10 uint32_t CACHE::llc_find_victim(uint32_t cpu, uint64_t instr_id, uint32_t set, const
    BLOCK *current_set, uint64_t ip, uint64_t full_addr, uint32_t type)
11 {
12     // fill invalid line first
13     for (uint32_t way=0; way<NUM_WAY; way++)
14         if (block[set][way].valid == false)
15             return way;
16
17     // MRU victim: the block at the most-recently-used position
18     for (uint32_t way=0; way<NUM_WAY; way++)
19         if (block[set][way].lru == 0)
20             return way;
21
22     assert(0);
23     return 0;
24 }
25
26 void CACHE::llc_update_replacement_state(uint32_t cpu, uint32_t set, uint32_t way,
    uint64_t full_addr, uint64_t ip, uint64_t victim_addr, uint32_t type, uint8_t hit)
27 {
28     if (hit && (type == WRITEBACK))
29         return;
30     // maintain the same recency stack as LRU; only the victim choice differs
31     return lru_update(set, way);
32 }
33
34 void CACHE::llc_replacement_final_stats()

```



```

35 {
36 }

```

D random.llc_repl

```

1 #include "cache.h"
2 #include <cstdlib>
3 // Random replacement: evict a uniformly random valid block.
4
5 void CACHE::llc_initialize_replacement()
6 {
7     cout << "Initialize LLC Random replacement policy" << endl;
8     srand(12345); // fixed seed for reproducibility
9 }
10
11 uint32_t CACHE::llc_find_victim(uint32_t cpu, uint64_t instr_id, uint32_t set, const
12     BLOCK *current_set, uint64_t ip, uint64_t full_addr, uint32_t type)
13 {
14     // fill invalid line first
15     for (uint32_t way=0; way<NUM_WAY; way++)
16         if (block[set][way].valid == false)
17             return way;
18
19     // pick a random way
20     return (uint32_t)(rand() % NUM_WAY);
21 }
22
23 void CACHE::llc_update_replacement_state(uint32_t cpu, uint32_t set, uint32_t way,
24     uint64_t full_addr, uint64_t ip, uint64_t victim_addr, uint32_t type, uint8_t hit)
25 {
26     // Random needs no recency state
27     return;
28 }
29
30 void CACHE::llc_replacement_final_stats()
31 {
32 }

```

E hcit.llc_repl

```

1 #include "cache.h"
2 #include <cstdlib>
3
4 // =====
5 // HCIT: Hard-to-Cache Identification Table
6 // Inspired by Bullseye (CBP 2025): a small subset of PCs accounts for
7 // most misses. Track per-PC cache behaviour, and spend protection
8 // resources only on the identified "hard-to-cache" tail.
9 // =====
10
11 #define maxRRPV 3
12 uint32_t rrpv[LLC_SET][LLC_WAY];
13
14 // --- Component 1: Hard-to-Cache Identification Table ---
15 #define HCIT_SETS 64
16 #define HCIT_WAYS 8
17 #define HCIT_ENTRIES (HCIT_SETS*HCIT_WAYS)
18
19 struct hcit_entry_t {
20     uint64_t tag;
21     uint32_t accesses;
22     uint32_t misses;
23     uint32_t inserts; // blocks inserted by this PC that were evicted
24     uint32_t reused; // of those, how many saw >=1 hit
25     uint8_t valid;
26     uint8_t protected; // currently receiving preferential treatment

```

```

27     uint32_t lru;
28 };
29 hcit_entry_t hcit[HCIT_SETS][HCIT_WAYS];
30 uint32_t hcit_clock;
31 uint32_t n_protected;
32
33 // which PC inserted each block (for reuse attribution)
34 uint64_t block_pc[LLC_SET][LLC_WAY];
35 uint8_t block_prot[LLC_SET][LLC_WAY];
36 uint8_t block_hit[LLC_SET][LLC_WAY]; // was this block reused before eviction?
37
38 // --- Component 3: adaptive admission thresholds ---
39 #define ACCESS_BASE 32
40 #define ACCESS_PER_PROT 8
41 #define MISS_BASE 16
42 #define MISS_PER_PROT 4
43 #define MAX_PROTECTED 32
44
45 // Reuse floor rises as the protected population grows: protection is a
46 // scarce resource, so later admissions must prove more reuse to earn it.
47 static inline double reuse_floor(uint32_t N) {
48     if (N < 8) return 0.08 + 0.01 * N;
49     if (N < 24) return 0.16 + 0.01 * (N - 8);
50     return 0.32;
51 }
52 #define TRAIN_MIN 8 // evicted samples needed before judging a PC
53
54 // stats
55 uint64_t stat_prot_inserts, stat_norm_inserts, stat_prot_hits, stat_admissions,
56     stat_evictions;
57
58 static inline uint32_t hcit_set(uint64_t ip) { return (ip >> 2) & (HCIT_SETS-1); }
59 static inline uint64_t hcit_tag(uint64_t ip) { return (ip >> 2) >> 6; }
60
61 static hcit_entry_t* hcit_lookup(uint64_t ip, bool allocate)
62 {
63     if (!ip) return NULL;
64     uint32_t s = hcit_set(ip);
65     uint64_t t = hcit_tag(ip);
66     for (int w=0; w<HCIT_WAYS; w++)
67         if (hcit[s][w].valid && hcit[s][w].tag == t) {
68             hcit[s][w].lru = ++hcit_clock;
69             return &hcit[s][w];
70         }
71     if (!allocate) return NULL;
72
73     int victim = 0; uint32_t oldest = 0xFFFFFFFF;
74     for (int w=0; w<HCIT_WAYS; w++) {
75         if (!hcit[s][w].valid) { victim = w; break; }
76         if (hcit[s][w].lru < oldest) { oldest = hcit[s][w].lru; victim = w; }
77     }
78     if (hcit[s][victim].valid && hcit[s][victim].protected_) {
79         n_protected--;
80         stat_evictions++;
81     }
82     memset(&hcit[s][victim], 0, sizeof(hcit_entry_t));
83     hcit[s][victim].tag = t;
84     hcit[s][victim].valid = 1;
85     hcit[s][victim].lru = ++hcit_clock;
86     return &hcit[s][victim];
87 }
88
89 void CACHE::llc_initialize_replacement()
90 {
91     cout << "Initialize LLC HCIT replacement policy "
92         << "(Hard-to-Cache Identification Table, adaptive thresholds)" << endl;
93     for (int i=0; i<LLC_SET; i++)
94         for (int j=0; j<LLC_WAY; j++) {
95             rrpv[i][j] = maxRRPV;
96             block_pc[i][j] = 0;
97             block_prot[i][j] = 0;
98             block_hit[i][j] = 0;
99         }

```

```

99     memset(hcit, 0, sizeof(hcit));
100     hcit_clock = 0; n_protected = 0;
101     stat_prot_inserts = stat_norm_inserts = stat_prot_hits = 0;
102     stat_admissions = stat_evictions = 0;
103 }
104
105 // --- Component 2: adaptive protection at victim selection ---
106 uint32_t CACHE::llc_find_victim(uint32_t cpu, uint64_t instr_id, uint32_t set, const
107     BLOCK *current_set, uint64_t ip, uint64_t full_addr, uint32_t type)
108 {
109     // prefer an unprotected maxRRPV line; fall back to any maxRRPV line
110     for (int pass=0; pass<2; pass++) {
111         int guard = 0;
112         while (guard++ < 64) {
113             for (int i=0; i<LLC_WAY; i++)
114                 if (rrpv[set][i] == maxRRPV && (pass == 1 || !block_prot[set][i]))
115                     return i;
116             for (int i=0; i<LLC_WAY; i++)
117                 if (rrpv[set][i] < maxRRPV) rrpv[set][i]++;
118         }
119     }
120     return 0;
121 }
122
123 void CACHE::llc_update_replacement_state(uint32_t cpu, uint32_t set, uint32_t way,
124     uint64_t full_addr, uint64_t ip, uint64_t victim_addr, uint32_t type, uint8_t hit)
125 {
126     if (hit && (type == WRITEBACK)) return;
127
128     if (hit) {
129         rrpv[set][way] = 0;
130         block_hit[set][way] = 1; // observed reuse
131         if (block_prot[set][way]) stat_prot_hits++;
132         hcit_entry_t *e = hcit_lookup(ip, true);
133         if (e) e->accesses++;
134         return;
135     }
136
137     // ---- eviction training: the outgoing block teaches its owner PC ----
138     if (block_pc[set][way]) {
139         hcit_entry_t *owner = hcit_lookup(block_pc[set][way], false);
140         if (owner) {
141             owner->inserts++;
142             if (block_hit[set][way]) owner->reused++;
143         }
144     }
145
146     // ---- miss / fill path ----
147     hcit_entry_t *e = hcit_lookup(ip, true);
148     if (!e) {
149         rrpv[set][way] = maxRRPV-1;
150         block_pc[set][way] = 0;
151         block_prot[set][way] = 0;
152         stat_norm_inserts++;
153         return;
154     }
155
156     e->accesses++;
157     e->misses++;
158
159     // adaptive admission: thresholds tighten as protection becomes scarce
160     uint32_t acc_th = ACCESS_BASE + ACCESS_PER_PROT * n_protected;
161     uint32_t miss_th = MISS_BASE + MISS_PER_PROT * n_protected;
162
163     // A PC is "hard-to-cache" only if it BOTH misses a lot (it matters)
164     // AND shows reuse (protection can actually help). High-miss/no-reuse
165     // PCs are streaming -- protecting them pollutes the cache.
166     if (!e->protected_ && n_protected < MAX_PROTECTED &&
167         e->accesses >= acc_th && e->misses >= miss_th &&
168         e->inserts >= TRAIN_MIN) {
169         double reuse_rate = (double)e->reused / (double)e->inserts;
170         if (reuse_rate >= reuse_floor(n_protected)) {
171             e->protected_ = 1;

```

```

170         n_protected++;
171         stat_admissions++;
172     }
173 }
174
175 // adaptive demotion: a protected PC that stops showing reuse loses status
176 if (e->protected_ && e->inserts >= TRAIN_MIN * 4) {
177     double reuse_rate = (double)e->reused / (double)e->inserts;
178     // demote well below the admission floor (hysteresis prevents thrash)
179     if (reuse_rate < reuse_floor(n_protected) * 0.5) {
180         e->protected_ = 0;
181         n_protected--;
182         stat_evictions++;
183     }
184 }
185
186 // protection = long-life insertion; everything else = short-life
187 if (e->protected_) {
188     rrpv[set][way] = 0;
189     block_prot[set][way] = 1;
190     stat_prot_inserts++;
191 } else {
192     // not yet identified as hard-to-cache -> standard SRRIP insertion
193     rrpv[set][way] = maxRRPV-1;
194     block_prot[set][way] = 0;
195     stat_norm_inserts++;
196 }
197 block_pc[set][way] = ip;
198 block_hit[set][way] = 0;
199 }
200
201 void CACHE::llc_replacement_final_stats()
202 {
203     cout << endl << "=== HCIT Replacement Policy Stats ===" << endl;
204     cout << "HCIT admissions:          " << stat_admissions << endl;
205     cout << "HCIT demotions:          " << stat_evictions << endl;
206     cout << "Currently protected:     " << n_protected << endl;
207     cout << "Protected inserts:       " << stat_prot_inserts << endl;
208     cout << "Normal inserts:          " << stat_norm_inserts << endl;
209     cout << "Hits on protected blks:  " << stat_prot_hits << endl;
210 }

```

F random_trace_gen.py

```

1  #!/usr/bin/env python3
2  """Randomly selects 4 traces from the assignment's 15-trace list for 4-core simulation.
3  """
4
5  import random, sys, json, datetime
6
7  TRACES = [
8      "401.bzip2-277B", "403.gcc-16B", "434.zeusmp-10B", "437.leslie3d-273B",
9      "450.soplex-92B", "456.hmmer-327B", "462.libquantum-1343B", "482.sphinx3-1522B",
10     "605.mcf_s-1644B", "605.mcf_s-665B", "619.lbm_s-3766B", "620.omnetpp_s-874B",
11     "621.wrf_s-8100B", "623.xalancbmk_s-700B", "628.pop2_s-17B",
12 ]
13
14 seed = int(sys.argv[1]) if len(sys.argv) > 1 else 42
15 random.seed(seed)
16 selected = random.sample(TRACES, 4)
17
18 print("Random Trace Generator for 4-Core Simulation")
19 print("Pool size:", len(TRACES))
20 print("Seed:", seed)
21 print("\nSelected traces:")
22 for i, t in enumerate(selected):
23     print("  Core %d: %s.champsimtrace.xz" % (i, t))
24
25 with open("selected_traces.txt", "w") as f:
26     f.write(" ".join(t + ".champsimtrace.xz" for t in selected) + "\n")

```

```
26 with open("selection_log.json", "w") as f:
27     json.dump({
28         "seed": seed,
29         "timestamp": str(datetime.datetime.now()),
30         "pool": TRACES,
31         "selected": selected
32     }, f, indent=2)
33
34 print("\nWrote selected_traces.txt and selection_log.json")
```